

SFA — Market Index (מחירון / OrganicMarketAgent): Feature Catalog

Document type: Product information — Market Index deep dive **Date:** 2026-06-03 **Status:** Live product (daily data push, active collectors) **Audience:** NotebookLM research corpus; PM planning; GTM research

Abstract

The SFA Market Index (מחירון) is a transparent, community-sourced price index for organic produce in Israel. Powered by the OrganicMarketAgent (OMA) Python pipeline, it collects price observations from multiple Israeli farm-shop and community sources, normalizes them against a catalog of ~65 products, computes rolling averages and ranges within a 7-day freshness window, and publishes the result to sfa.nimrod.bio/market/ with explicit data-quality and freshness signals. The index is honest about its purpose: it answers "what do things roughly cost at the source?" — not a real-time ticker and not a replacement for any single seller's pricing. A mandatory four-part disclaimer on every market page makes this explicit. The UX offers a cards-and-table density toggle, category chip filters, freshness pills on every product, and a per-product detail page with a 7-day and 28-day price history graph.

1. Purpose and scope

1.1 What the Market Index is

The Market Index provides a **community price transparency signal** for organic vegetables in Israel. It answers a single question: *what do things roughly cost at the source?* — by aggregating price observations from farms, CSAs, farm shops, and farmers markets, without replacing any single seller's pricing.

1.2 What the Market Index is not

The mandatory disclaimer on every market page (the `.mkt-disc` component, never suppressible) states four clarifying points:

Clause	Hebrew label	Meaning
What	מה	Rolling community average from multiple sources — not a single seller's price
Where	מאיפה	Community farms, farm shops, CSAs, and farmers markets
Why	למה	Price transparency for the organic community — not financial advice
Not	לא	Not a replacement for negotiation, not a guaranteed price, not real-time

This disclaimer is a design requirement, never hidden or suppressed.

1.3 Product scope (V1)

- **In scope:** Organic **vegetables** in **community** sales channels; baskets/CSA as first-class basket products (not decomposed to per-kg in V1)
- **Explicitly out of scope:** Donations, cleaning products, dry-grocery lines on mixed retail grids (handled via catalog scope skip rules, not silent deletion)
- **Geography:** Israeli market channels
- **Language:** Hebrew product names (name_he) — the system explicitly handles Hebrew NLP normalization

2. Data collection: sources and collectors

2.1 Source architecture

The OMA pipeline uses a multi-source collection architecture. Each source has a trust tier, a source ID (SRC_XX format), and an associated collector.

Current active sources include:

Source ID	Type	Collector	Notes
SRC_MP01	MyPIPS farm shop	MypipsCollector (Playwright)	mashtelatharoe
SRC_MP02	MyPIPS farm shop	MypipsCollector (Playwright)	anatiyot
SRC_MP03	MyPIPS farm shop	MypipsCollector (Playwright)	fruit4soul
SRC_MP04	MyPIPS farm shop	MypipsCollector (Playwright)	finerotem
(others)	Community channels	HTTP/scrapper collectors	Various farm + CSA sources

Total sources: 25 registered; 7+ active at any given time (scraping health varies by source).

2.2 MyPIPS collectors (Playwright)

MyPIPS farm shops are the primary current community source. The `MypipsCollector` uses **headless Chromium via Playwright** (Python) to fetch product listings from `mypips.app/{handle}/products`. This is required because MyPIPS renders product data client-side (SPA), not in static HTML.

The collector: 1. Navigates to the shop's product listing page 2. Extracts product names (Hebrew), prices, and units from the rendered DOM 3. Writes raw items to `raw_extracted_items` with `status=extracted` 4. Applies the `include0rganic` filter (scoped to anatiyot only) for organic-certification filtering

2.3 Collection pipeline stages

```
Collect (HTTP fetch / Playwright scrape)
↓
Parse → raw_extracted_items (raw lines tied to source text)
↓
Normalize (catalog match, unit standardization, scope-skip rules)
↓
```

```
Aggregate (rolling windows, min-source thresholds, daily logic)
↓
Publish → sfa_ingest_push.py → POST /api/v1/ingest → uPress MySQL
```

Each stage is inspectable via the local Flask admin UI (127.0.0.1, not public).

2.4 Normalization

Normalization maps raw extracted product names to catalog products. It uses:

- **Catalog aliases** — 232 active aliases mapping variant product names to canonical products
- **Scope-skip rules** — 301 approved rules marking out-of-scope items (donations, dry grocery, cleaning products) as `ignored` with an explicit `ignore_reason_code=approved_scope_skip` rather than silent deletion
- **Hebrew NLP** — Hebrew product name matching with plural/construct-state awareness

Resolution rate: 100% (0 unresolvable items of 508 total, as of 2026-03-31 system health snapshot). All non-matching items are either normalized to catalog products or explicitly scope-skipped — nothing is silently dropped.

2.5 Data quality snapshot

The pipeline computes a `data_quality` snapshot (via `compute_raw_pipeline_counts`) that appears in:

- The local admin dashboard (funnel KPIs) - `public_report.json` (machine-readable) - `manifest.json` (quick consumers)

3. Rolling average and freshness model

3.1 Rolling window

The Market Index uses a **7-day rolling window** as the primary freshness signal. Price observations older than 7 days are still included in the database but are flagged as stale in the UI. The `PublishEngine` applies minimum-source thresholds and staleness rules before including a product in the public index.

3.2 Aggregates

For each published product, the index provides:

Metric	Description
<code>last_price / price_current</code>	Most recent observation in the window
<code>price_min</code>	Minimum observation in the rolling window
<code>price_max</code>	Maximum observation in the rolling window
<code>price_median</code>	Median observation in the rolling window
<code>source_count</code>	Number of distinct sources contributing in the window
<code>observation_count</code>	Total observation rows in the window
<code>freshness_days</code>	Days since the most recent observation

3.3 Freshness pills (LOCKED thresholds)

Every product card and detail page shows a **freshness pill** with color and label:

Condition	Class	Label (Hebrew)	Color
<code>freshness_days ≤ 3</code>	<code>fresh--fresh</code>	"ימים N טרי · עודכן היום" / "לפני"	Green (--gj-leaf)
<code>freshness_days 4–7</code>	<code>fresh--aging</code>	"ימים N מתעדכן · לפני"	Yellow (--gj-sun)
<code>freshness_days > 7</code>	<code>fresh--stale</code>	"ימים N ישן · לפני"	Red (--gj-tomato)
<code>freshness_days = null</code>	<code>fresh--stale</code>	"אין דיווח"	Red (--gj-tomato)

These thresholds are locked in the delivery-tier code (`market_list.php` , `market_product.php`) and match the legend shown in the toolbar.

3.4 `display_bucket` column

A `display_bucket` column on `raw_extracted_items` enables future price-range segmentation (e.g., budget / standard / premium) — infrastructure in place (migration 034) but not yet surfaced in V1 UX.

4. Honest empty and stale states

The Market Index never hides empty or stale data:

Empty product card (no price data): - Card shows "—" for the price - "אין דיווח" (no report) label - Card class `is-empty`

Stale product card (freshness > 7 days): - Card shows the price with a red "ישן" pill - Card class `is-stale` - Not removed from the index — kept visible to signal that a product exists but data is old

Empty market list (no products at all): - Shows an explicit "אין נתוני מחיר — חזרו בקרוב או תרמו מחיר" (no price data — come back soon or contribute a price) message with a mailbox icon

These states are first-class design requirements in the delivery-tier templates.

5. Market UX

5.1 Market list page (/market/)

Components: - Mandatory disclaimer (`.mkt-disc`) — always first, never suppressible - Category filter chips (`.fchips`) — filter by product category (vegetables, herbs, fruits, etc.) - Freshness legend — color-coded guide to the three freshness states - Cards ⇌ Table toggle — density switch between card grid and compact table view - Product count display — number of products shown

Cards view (default): - Product card (`.pcard`) per product - Shows: Hebrew product name, unit label (לק"ג / ליחידה / לאגודה), current price, price range (min–max), source count, sparkline stub (7-bar; full sparkline only on detail page), freshness pill - Cards link to the product detail page

Table view: - Compact rows with same data fields in column format - Better for comparing multiple products at once

5.2 Market detail page (/market/{slug})

Components: - Compact mandatory disclaimer (shorter form, still always present) - Product hero block: Hebrew name, English name, current price (large), unit label, freshness pill - Price range: min-max range, source count, observation count - **Price history graph** (`.pgraph`): SVG line chart built from `history[]` array - Range selector: **7 days (live)**, **28 days (live)**, 90 days (disabled — `.is-disabled`), year (disabled) - 7-day and 28-day ranges are active; 90-day and year-range are disabled server-side pending aggregate data - Price stats strip: min / max / median labeled values - Provenance: source list showing contributing sources - Cross-link: if a `crop_book` entry exists for this product, shows a link to the Crop Book page - Contribute prompt: "contribute a price observation" CTA

5.3 Price history graph details

The history graph is built as a pure SVG path from `history[]` rows (ordered oldest-to-newest for left-to-right rendering): - X axis: time (normalized across the selected window) - Y axis: price (ILS), normalized to SVG coordinate space - Points plotted at each observation date - The 28-day range is the maximum live range; 90d/year are disabled with explicit `.is-disabled` class on the range selector buttons (not hidden — users can see what is planned but not yet available)

6. Current limitations and known gaps

6.1 Aggregate data pending

The 90-day and year-range history views are disabled (`is-disabled` server-side). They require the aggregation pipeline to compute longer-window rolling statistics from the full observation history. This is tracked as a future work item, not yet specced.

6.2 Freshness dependency on scraper health

The 7-day freshness model depends on scrapers running successfully every day. MyPIPS Playwright collectors require a live Chromium environment on waldhomeserver. Scraper outages (e.g., network issues, site changes) will age data and surface "ישן" pills — which is the correct honest behavior, not a bug.

6.3 Price data is community-sourced, not exhaustive

The index covers community channels (farm shops, CSAs, markets). It does not cover supermarket pricing, wholesale markets, or private farm direct sales. Source count on each product card (e.g., "3 מקורות") is explicit — users can see how many sources contributed.

6.4 Sparkline on list view

The sparkline (7-bar price trend indicator) on the product list card is currently an empty placeholder (`spark--empty`) because per-day sparkline data is only available in the detail page's `history[]`

payload, not the list view query. The list-view sparkline is intentionally degraded to avoid fetching heavy history data for every card.

7. Data push and publish pipeline

7.1 Publish flow

1. OMA pipeline on walddhomeserver runs collection → normalization → aggregation
2. PublishEngine builds the public product list and computes data_quality snapshot
3. sfa_ingest_push.py sends a POST https://sfa.nimrod.bio/api/v1/ingest request with HMAC-SHA256 signature
4. The uPress ingest endpoint writes the payload into the MySQL read-mirror
5. The delivery-tier PHP reads the MySQL mirror and renders the market pages

A daily cron runs at 06:00 UTC on walddhomeserver. An anti-drift freshness guard (freshness_guard.py) runs at 06:45 UTC and re-triggers the ingest push if the sfa.nimrod.bio data is stale.

7.2 Data schema on uPress MySQL

The uPress MySQL read-mirror (sfa-mysql-mirror.md schema) holds: - Products with name_he, slug, unit_he, category - Observations with price, date, source_id - Rolling aggregates (price_current, price_min, price_max, price_median, freshness_days, source_count, observation_count) - History arrays for the detail page graph

The MySQL side is a **read-mirror** — the canonical data lives in the PostgreSQL instance on walddhomeserver.

7.3 Historical publish tiers (superseded)

The original publish path (S002 era and before) used WordPress WP REST API to upload static JSON files to www.nimrod.bio (now retired). The current path (since 2026-05-28) uses direct HMAC-signed ingest to sfa.nimrod.bio/api/v1/ingest. The WordPress path is permanently retired; references to wp_upload.py , ftps_upload.py , or upload_dispatch.py describe the retired tier.

8. Admin and transparency infrastructure

8.1 Local admin UI

A local Flask admin UI (127.0.0.1, not public-facing) provides: - **Dashboard** — funnel KPIs (collected → parsed → normalized → published counts) - **Scope-skip catalog** (/catalog/scope-skip) — inspectable list of 301 approved scope-skip rules with reason codes - **Catalog tools** — alias management (232 active aliases) - **Run management** — trigger manual ingestion runs, view run logs - **Maintenance actions** — re-normalize, re-aggregate

8.2 Data quality in the public payload

The `public_report.json` (pushed via ingest API) carries a `data_quality` block with: - Pipeline resolution counts (extracted / normalized / unresolvable / ignored) - Source health (active sources count, stale sources) - `artifact_version` timestamp

This block is visible to technical consumers and is logged for monitoring.

8.3 Transparency in the UI

Beyond the mandatory disclaimer, the detail page shows: - Source count (how many sources contributed this product's price) - Observation count (how many individual observations) - "Updated" date (when the data was last refreshed) - Provenance list (which sources contributed)

9. System health metrics (reference snapshot, 2026-03-31)

Metric	Value
Resolution rate	100%
Products in catalog	67 (62 with observations at snapshot)
Active source count	7 of 25 registered
Active aliases	232
Scope-skip rules	301
Alembic migrations (market schema)	034 (display_bucket; market-relevant head)
Tests (full suite)	127+ passed (plus crop book suite)

10. Future directions

- **90-day / year-range aggregates** — requires extending the aggregation pipeline to maintain longer-window statistics; detail page range selector is already built, waiting on data
- **Price contribution flow** — the community page's "contribute a price" CTA links to a request form; a structured price-contribution pipeline (UC trust class) is designed but not yet a live data path
- **More community sources** — additional MyPIPS shops and direct farm data sources are candidates for the next source expansion
- **Market x Crop Book cross-linking** — the detail page already links to the Crop Book entry when a `book_slug` exists; richer integration (e.g., documented price comparison on the crop calculator) is planned for Crop Book v1 calculator #9

Sources: *PROJECT_VISION_AND_SYSTEM_MAP.md*, *_aos/context/PROJECT_CONTEXT.md*, *_aos/roadmap.yaml* (S002 program notes), *sfa_delivery/templates/pages/market_list.php*, *sfa_delivery/templates/pages/market_product.php*, *_aos/work_packages/S002/SFA-S002-P001-WP002/LOD400_spec.md* (MyPIPS integration), *organic_market_agent/publisher/sfa_ingest_push.py*.